

CLEAN ARCHITECTURE

SERVER-FIRST ISLANDS

MODULAR DATA STORES

SCALABLE STRUCTURE

# CODEBASE ARCHITECTURE & PROJECT STRUCTURE

File Map, Boundaries & Scalability Specifications — Mohammed Sarook Mohammed Safith ("SAFITH THE VIBE CODER 🇵🇰")

ROUTING

Next.js 14+ App Router

COMPONENTS

Server & Client Islands

DATA ARCHITECTURE

Decoupled Static Stores

TYPE SAFETY

TypeScript Strict Contracts

## 1 Visual Directory Hierarchy Map

```
portfolio/
├─ package.json      # Dependency list & build scripts (Next.js, Tailwind, Framer Motion)
├─ tsconfig.json     # Strict TypeScript compiler options & `@/*` path aliases
├─ tailwind.config.ts # CSS custom property bindings & design tokens
├─ next.config.mjs   # Next.js compiler tuning & next-intl plugin integration
├─
├─ messages/        # i18n Translation dictionaries (en.json, ta.json, ur.json)
├─ public/           # Static assets: CV PDF, project preview screenshots, avatar
├─
├─ src/              # Application Source
│  ├─ middleware.ts  # Edge locale interceptor (handles /[locale]/...)
│  ├─ app/           # App Router root, globals.css, and localized routes
│  ├─ components/    # layout/, sections/, ui/, motion/ components
│  ├─ data/          # Static data stores: projects.ts, skills.ts, timeline.ts
│  ├─ hooks/         # Custom hooks: useTheme.ts, useScrollSpy.ts
│  ├─ lib/           # Utilities: cn() helper, theme configs
│  └─ types/         # Strict TypeScript interfaces (index.ts)
```

## 2 Component Boundaries: Server (RSC) vs. Client

Next.js 14+ defaults to Server Components (RSC). Client Components ( `'use client'` ) are strictly isolated to interactive islands:

### Server Components (Zero Client JS)

- Runs at build time (SSG); outputs fast HTML.
- `[locale]/layout.tsx` (HTML structure, fonts)
- `[locale]/page.tsx` (Assembles all 7 sections)
- `Footer.tsx` (Static links, copyright)
- `AboutSection.tsx` (Static biography, metrics)
- `ReferencesSection.tsx` (Degree notes)

### Client Components ('use client')

- Runs in browser; handles state & events.
- `Header.tsx` (Sticky scroll observer)
- `ThemeToggle.tsx` (4-theme selector)
- `LanguageToggle.tsx` (URL locale routing)
- `TypedText.tsx` (Animated role cycler)
- `ContactForm.tsx` (Validation & Formspree POST)
- `ScrollReveal.tsx` (Framer Motion viewport inView)

### 3 Data Flow: Decoupled Stores vs. UI Plates

#### The Restaurant Menu Analogy

In a restaurant, the **Menu** ( `src/data/projects.ts` ) holds the food items and prices. The **Plates** ( `src/components/ui/ProjectCard.tsx` ) are the visual presentation. When you add a new project or change a description, you simply update the data array without risking breaking the UI design.

| DATA STORE                        | TYPE CONTRACT                | CONSUMING SECTION COMPONENT            | DELIVERED CAPABILITY                                         |
|-----------------------------------|------------------------------|----------------------------------------|--------------------------------------------------------------|
| <code>src/data/projects.ts</code> | <code>Project[]</code>       | <code>&lt;ProjectsSection /&gt;</code> | Renders 4 project cards with tags, statuses, and live links. |
| <code>src/data/skills.ts</code>   | <code>SkillItem[]</code>     | <code>&lt;SkillsSection /&gt;</code>   | Renders grouped badges across Languages, Frontend, QA, etc.  |
| <code>src/data/timeline.ts</code> | <code>TimelineEntry[]</code> | <code>&lt;TimelineSection /&gt;</code> | Renders chronological degree and internship milestones.      |

### 4 Scalability: How to Extend the Codebase

| EXTENSION GOAL     | FILES TO MODIFY                                     | EXECUTION STEPS                                                                                                                                 |
|--------------------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Add a 5th Project  | <code>src/data/projects.ts</code>                   | Append a new project object. The card and <code>/projects/[slug]</code> page are generated automatically at build time.                         |
| Add a 5th Theme    | <code>globals.css</code> , <code>themes.ts</code>   | Declare <code>html[data-theme="new-name"]</code> CSS variables and register the name in <code>themes.ts</code> . Appears in toggle immediately. |
| Add a New Language | <code>messages/</code> , <code>middleware.ts</code> | Add <code>messages/[locale].json</code> , register the code in middleware, and add the option to <code>LanguageToggle.tsx</code> .              |